

**Министерство высшего образования и науки Республики Казахстан
Северо-Казахстанский университет им. М. Козыбаева
Кафедра «Информационно-коммуникационные технологии»**

**Объектно-ориентированное программирование
МОДУЛЬ 1**

**«Объектная модель учёта пациентов и медицинских приёмов в
больнице»**

Выполнили студенты группы ИСУ-24-1:
Бахтагареев М.Р.
Добош Н.А.

Проверила: Астапенко Н.В.

Петропавловск, 2026

Содержание

1. Введение	3
1.1. Цель работы	3
1.2. Задачи работы	3
2. Анализ предметной области	4
2.1. Границы предметной области	4
2.2. Основные классы и их ответственность	4
2.3. Атрибуты и методы	5
2.4. Иерархия наследования	6
3. Построение объектной модели	6
3.1. Диаграмма классов	6
3.2. Связи между классами	7
3.3. Обоснование типов связей	8
3.4. Кратности	9
3.5. Сценарий работы модели	9
4. Python-реализация	10
4.1. Базовые классы и наследование	10
4.2. Полиморфизм	10
4.3. Композиция и агрегация	11
4.4. Ассоциации	11
4.5. Демонстрация	12
5. Вопросы для самопроверки	13
6. Заключение	14

1. Введение

1.1. Цель работы

Цель работы - построить объектную модель учёта пациентов и медицинских приёмов в больнице на основе выделенных объектов предметной области. Модель должна показать свойства объектов, их поведение, наследование и связи между самостоятельными и составными частями системы.

1.2. Задачи работы

1. выделить основные объекты выбранной предметной области и представить их в виде классов;
2. определить свойства и методы каждого класса на основе реальной программной реализации;
3. построить диаграмму классов с использованием ассоциации, агрегации, композиции и обобщения там, где эти отношения соответствуют модели;
4. объяснить выбор типов связей и кратностей;
5. подтвердить согласованность модели с программным кодом и демонстрационным сценарием.

В качестве источника истины для реализации использован исходный Python-проект. Текущий отчёт из материалов архива рассмотрен как черновик: его формулировки проверены по фактическому коду и скорректированы там, где они расходились с реализацией.

2. Анализ предметной области

2.1. Границы предметной области

Рассматриваемая модель охватывает конкретный участок работы больницы: регистрацию пациента, ведение медицинской карты, запись на приём к врачу, проведение приёма, фиксацию диагноза и назначения, получение истории обращений, а также организацию отделений и медицинского персонала. Полная информационная система больницы не моделируется.

Процесс со стороны пациента связан с медицинской частью модели, а организационная структура представлена больницей, отделениями и персоналом. Класс Appointment является центральным событием медицинского процесса: он хранит дату и время, пациента, врача и результат приёма.

2.2. Основные классы и их ответственность

Класс	Состояние / свойства	Ответственность
Person	Общие данные человека: ФИО, дата рождения, телефон.	Общий родитель для Patient и MedicalStaff.
Patient	Полис, медицинская карта.	Оформляет карту, создаёт записи на приём, отменяет свою запись, получает историю.
MedicalStaff	Табельный номер и стаж.	Общий тип для медицинского персонала и единый интерфейс get_info().
Doctor	Специальность, категория, проведённые приёмы.	Проводит приём и сохраняет проведённые приёмы.
Nurse	Профиль работы и смена.	Готовит кабинет и сопровождает пациента.
Hospital	Название, адрес, отделения.	Создаёт отделения, ищет врачей и собирает статистику.
Department	Название, расположение, закреплённый персонал.	Принимает внешние объекты персонала и формирует расписание.
MedicalCard	Номер, дата заведения, записи о приёмах.	Хранит историю обращений пациента.
Appointment	Дата/время, пациент, врач, статус, диагноз, назначения.	Переносит, завершает или отменяет приём; формирует его описание.

Таким образом, в модель включены девять классов, которые непосредственно участвуют в выбранном процессе и организационной структуре. Дополнительные сущности, не используемые проектом, не вводились.

2.3. Атрибуты и методы

Класс	Атрибуты, зафиксированные в коде	Методы и свойства
Person	<code>_full_name: str; _birth_date: date; _phone: str</code>	<code>full_name; get_age(); get_contacts()</code>
Patient	<code>_insurance_no: str; _card: MedicalCard</code>	<code>insurance_no; card; issue_card(); make_appointment(); cancel_appointment(); get_history()</code>
MedicalStaff	<code>_staff_no: str; _experience: int</code>	<code>get_info()</code>
Doctor	<code>_specialty: str; _category: str; _appointments: list</code>	<code>specialty; get_info(); conduct_appointment(); appointments</code>
Nurse	<code>_profile: str; _shift: str</code>	<code>get_info(); prepare_room(); escort_patient()</code>
Hospital	<code>_name: str; _address: str; _departments: list</code>	<code>name; add_department(); find_doctors(); get_statistics()</code>
Department	<code>_name: str; _location: str; _staff: list</code>	<code>name; location; attach_staff(); schedule(); staff</code>
MedicalCard	<code>_number: str; _issue_date: date; _records: list</code>	<code>number; add_record(); get_history()</code>
Appointment	<code>_when: datetime; _patient: Patient; _doctor: Doctor; _status: str; _diagnosis: str/None; _prescription: str/None</code>	<code>reschedule(); complete(); cancel(); describe()</code>

В таблицу включены именно фактические атрибуты и методы проекта. Например, у `Doctor` нет отдельных методов `поставитьДиагноз()` и `выписатьНазначение()`: в реальном коде эти данные передаются в `conduct_appointment()`, который вызывает `complete()` у `Appointment`.

Инкапсуляция

Внутренние данные хранятся в атрибутах с ведущим подчёркиванием. Для части значений предусмотрены свойства только для чтения, например `full_name`, `specialty`, `appointments`, `staff` и `card`. При этом подчёркивание в Python является соглашением об уровне доступа, а не жёстким запретом изменения, поэтому в отчёте это не называется абсолютной защитой.

2.4. Иерархия наследования

Класс Person содержит общие данные и поведение для пациента и медицинского персонала. MedicalStaff является специализированным наследником Person и объединяет общие данные сотрудников. Doctor и Nurse наследуются от MedicalStaff и переопределяют get_info(), поэтому один и тот же вызов из Department.schedule() работает с объектами обоих классов.

3. Построение объектной модели

3.1. Диаграмма классов

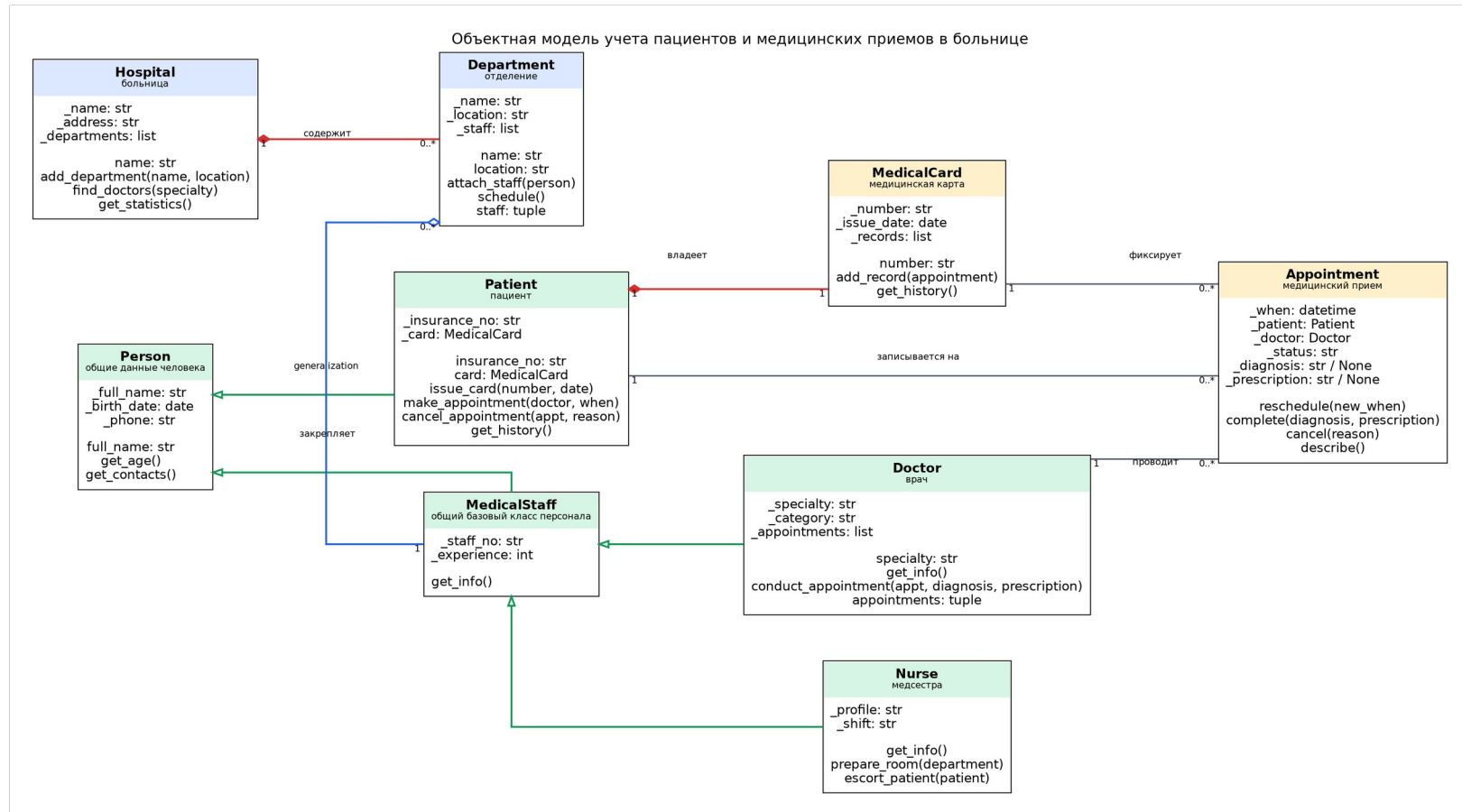


Рисунок 1 - Диаграмма классов предметной области «Учёт пациентов и медицинских приёмов в больнице»

На диаграмме показаны четыре разрешённых в работе типа отношений: обобщение, композиция, агрегация и ассоциация. Связь Dependency и пунктирные стрелки не используются.

3.2. Связи между классами

Связи выбраны по тому, как объекты создаются, хранятся и взаимодействуют в коде. Ниже приведены все отношения между классами, используемые в модели.

Пара классов	Тип	Кратность	Основание в реализации
Hospital - Department	Композиция	1 : 0..*	Department создаётся внутри Hospital.add_department() и хранится в _departments.
Patient - MedicalCard	Композиция	1 : 1	Patient хранит единственную карту, а после исправления карта создаётся непосредственно в issue_card().
Department - MedicalStaff	Агрегация	1 : 0..*	Сотрудники создаются отдельно и передаются в attach_staff(), поэтому их жизненный цикл не принадлежит отделению.
Patient - Appointment	Ассоциация	1 : 0..*	Appointment хранит ссылку на пациента; пациент создаёт приём через make_appointment().
Doctor - Appointment	Ассоциация	1 : 0..*	Appointment хранит ссылку на врача, а Doctor хранит проведённые приёмы в _appointments.

3.3. Обоснование типов связей

Композиция применена к Hospital - Department, потому что отделение входит в структуру конкретной больницы и создаётся её методом add_department(). Композиция Patient - MedicalCard отражает единственную карту пациента; в окончательной версии кода она создаётся внутри issue_card(), что устраняет расхождение между обозначенной связью и способом создания объекта.

Агрегация Department - MedicalStaff показывает связь «отделение - закреплённый персонал»: объекты Doctor и Nurse сначала создаются отдельно, затем добавляются в отделение через attach_staff(). Принадлежность к отделению не является условием существования сотрудника.

3.2. Связи между классами (продолжение)

Пара классов	Тип	Кратность	Основание в реализации
MedicalCard - Appointment	Ассоциация	1 : 0..*	Завершённый Appointment добавляется в _records медицинской карты методом add_record().
Person -> Patient	Обобщение	-	Patient получает общие данные и методы Person.
Person -> MedicalStaff	Обобщение	-	MedicalStaff получает общие данные и методы Person.
MedicalStaff -> Doctor	Обобщение	-	Doctor специализирует персонал полем specialty и поведением проведения приёма.
MedicalStaff -> Nurse	Обобщение	-	Nurse специализирует персонал профилем и сменой.

Всего в модели десять связей: четыре обобщения, две композиции, одна агрегация и три ассоциации. Количество и типы связей не подгонялись под формальную квоту: каждая связь имеет конкретное основание в коде и предметной области.

3.3. Обоснование ассоциаций и обобщений

Ассоциации применены между самостоятельными объектами. Appointment не управляет жизненным циклом пациента или врача, а MedicalCard не создаёт Appointment сам: он только принимает завершённую запись через add_record(). Поэтому для этих отношений ромб не используется.

Обобщение отражает отношение «является»: Patient является Person, MedicalStaff является Person, Doctor и Nurse являются MedicalStaff. Практический результат наследования виден в повторном использовании общих полей и методов, а также в полиморфном get_info().

3.4. Кратности

Связь	Кратность	Пояснение
Hospital - Department	1 : 0..*	Код допускает создание больницы без добавленного отделения, поэтому нижняя граница 0.
Patient - MedicalCard	1 : 1	Один пациент может иметь одну карту; повторный вызов <code>issue_card()</code> завершается ошибкой.
Department - MedicalStaff	1 : 0..*	<code>attach_staff()</code> допускает пустое отделение и не задаёт минимальное число сотрудников.
Patient - Appointment	1 : 0..*	Пациент может не иметь приёмов или иметь несколько.
Doctor - Appointment	1 : 0..*	Врач может не иметь проведённых приёмов или проводить множество приёмов.
MedicalCard - Appointment	1 : 0..*	В карте может не быть записей либо может быть несколько завершённых приёмов.

3.5. Сценарий работы модели

1. создаётся объект `Hospital`, затем через `add_department()` создаются терапевтическое и хирургическое отделения;
2. вне отделения создаются два `Doctor` и один `Nurse`, после чего они прикрепляются к отделениям через `attach_staff()`;
3. через общий `get_info()` отделение и `main.py` демонстрируют полиморфный вывод для врача и медсестры;
4. создаётся `Patient` и через `issue_card()` ему выдаётся единственная `MedicalCard`;
5. `patient.make_appointment()` создаёт `Appointment` с датой, пациентом и врачом; медсестра подготавливает кабинет;
6. `doctor.conduct_appointment()` завершает приём, записывает диагноз и назначения; `complete()` добавляет приём в `MedicalCard`;
7. для отдельного приёма демонстрируется перенос, а для другого - отмена; история, поиск врачей и статистика выводятся из созданных объектов.

4. Python-реализация

4.1. Базовые классы и наследование

В реализации общие признаки вынесены в Person, а общие признаки медицинского персонала - в MedicalStaff. Производные классы используют super() и добавляют собственные атрибуты и действия.

```
class Patient(Person):
    def __init__(self, full_name, birth_date, phone, insurance_no):
        super().__init__(full_name, birth_date, phone)
        self._insurance_no = insurance_no
        self._card = None

class Doctor(MedicalStaff):
    def get_info(self):
        return f"Врач {self._full_name} ({self._specialty})"

class Nurse(MedicalStaff):
    def get_info(self):
        return f"Медсестра {self._full_name} ({self._profile}, {self._shift} смена)"
```

4.2. Полиморфизм

Department.schedule() вызывает get_info() без отдельной ветки для Doctor и Nurse. Каждый наследник предоставляет собственную реализацию этого метода, поэтому общий код работает с разными типами объектов.

```
def schedule(self):
    return [s.get_info() for s in self._staff]

for person in (doctor, surgeon, nurse):
    print(person.get_info())
```

Результат демонстрации: для врача выводится строка с его специальностью, а для медсестры - с профилем и сменой. Это подтверждает фактический полиморфизм, а не только наличие одинакового имени метода.

4.3. Композиция и агрегация в коде

Композиция Hospital - Department реализована созданием Department внутри add_department(). Для Patient - MedicalCard окончательная версия использует тот же принцип: issue_card() создаёт объект MedicalCard и сохраняет его в пациенте.

```
def add_department(self, name, location):
    dept = Department(name, location)
    self._departments.append(dept)
    return dept

def issue_card(self, number, issue_date):
    if self._card is not None:
        raise ValueError("У пациента уже есть медицинская карта")
    self._card = MedicalCard(number, issue_date)
    return self._card
```

Агрегация Department - MedicalStaff реализована иначе: объект персонала создаётся до вызова attach_staff(), а затем сохраняется в списке отделения. Сам отделение не создаёт и не уничтожает сотрудника.

```
doctor = Doctor(...)
nurse = Nurse(...)
therapy.attach_staff(doctor)
therapy.attach_staff(nurse)
```

4.4. Ассоциации и состояние Appointment

Appointment хранит ссылки на пациента и врача. После завершения он добавляется в MedicalCard.add_record(), поэтому карта получает историю обращений. Методы reschedule(), complete() и cancel() проверяют допустимое состояние.

```
def reschedule(self, new_when):
    if self._status != "Запланирован":
        raise ValueError("Перенести можно только запланированный приём")
    self._when = new_when

def complete(self, diagnosis, prescription=None):
    if self._status != "запланирован":
        raise ValueError("Завершить можно только запланированный приём")
    self._status = "завершён"
    self._diagnosis = diagnosis
    self._prescription = prescription
    self._patient.card.add_record(self)
```

4.5. Демонстрация

Сценарий из main.py запускается без сторонних библиотек. В нём создаются все основные объекты, демонстрируются четыре типа отношений, полиморфизм, история обращений, перенос и отмена приёмов, поиск врачей и статистика больницы.

```
patient = Patient("Бахтаргеев М.Р.", date(2005, 4, 21),  
                 "+7 700 777-88-99", "123456789012")  
patient.issue_card("K-000123", date.today())  
  
appt = patient.make_appointment(doctor, datetime(2026, 9, 20, 10, 30))  
doctor.conduct_appointment(appt, "ОРВИ", "Постельный режим, обильное питьё")  
  
print(patient.get_history())  
print(hospital.find_doctors("терапевт"))  
print(hospital.get_statistics())
```

Фактический вывод ключевых результатов:

```
Записан: 20.09.2026 10:30 - врач Иванов А.С. (терапевт) - запланирован  
Завершён: диагноз ОРВИ; назначения сохранены  
История: 1 запись; терапевты: Иванов А.С.  
Статистика: отделений 2; сотрудников 3; врачей 2; медсестёр 1  
Полиморфизм: Doctor, Doctor, Nurse через get_info()
```

Дополнительно проверены недопустимые переходы состояния: отменённый и завершённый приём нельзя перенести, завершённый приём нельзя отменить. Эти проверки соответствуют логике методов Appointment после исправления исходной реализации.

5. Вопросы для самопроверки

5.1. Чем отличается агрегация от композиции?

Оба отношения описывают связь «целое - часть». При композиции часть принадлежит конкретному целому и её существование рассматривается как часть жизненного цикла целого. В модели так связаны Hospital и Department, а также Patient и MedicalCard. При агрегации объекты могут существовать отдельно и затем быть объединены в группу. Поэтому Department и MedicalStaff связаны агрегацией: сотрудник создаётся до прикрепления к отделению и не является собственностью отделения по жизненному циклу.

5.2. Что означает принцип наследования в объектной модели?

Наследование позволяет описать специализированный класс на основе общего родительского класса. Дочерний класс получает его доступное поведение и структуру, а затем может добавлять собственные свойства и переопределять методы. В проекте Patient и MedicalStaff наследуются от Person, а Doctor и Nurse - от MedicalStaff. Благодаря этому общие данные человека и персонала не дублируются.

5.3. Какое преимущество даёт использование объектной модели по сравнению с процедурным подходом?

Объектная модель объединяет состояние и действия, относящиеся к одной сущности, внутри класса. За счёт этого легче поддерживать границы ответственности: Appointment управляет состоянием приёма, MedicalCard хранит историю, Hospital отвечает за структуру больницы, а Doctor проводит приём. Наследование и полиморфизм дополнительно позволяют работать с общим типом, не повторяя одинаковую логику для каждого наследника.

6. Заключение

В результате анализа предметной области и фактического Python-кода согласована единая модель из девяти классов: Person, Patient, MedicalStaff, Doctor, Nurse, Hospital, Department, MedicalCard и Appointment. Для каждого класса определены реальные атрибуты и методы, а связи в UML сопоставлены с тем, как объекты создаются и взаимодействуют в программе.

Диаграмма отражает четыре используемых типа отношений: обобщение, композицию, агрегацию и ассоциацию. Кратности выбраны по фактическому поведению кода, поэтому для Hospital - Department и Department - MedicalStaff используется нижняя граница 0, а не искусственно заданное обязательное количество.

Проверка main.py показала согласованное выполнение сценария: создаются отделения и персонал, регистрируется пациент, оформляется медицинская карта, создаётся и завершается приём, результат попадает в историю, выполняются перенос и отмена, поиск врачей, статистика и полиморфный вывод.

Отдельно исправлены два несоответствия исходной реализации: создание MedicalCard теперь происходит внутри Patient.issue_card(), что соответствует композиции, а методы Appointment.reschedule() и cancel() корректно запрещают операции над приёмами, которые уже не находятся в статусе «запланирован».

Итоговая модель предназначена именно для учебного задания первого модуля и не расширяется до полной медицинской информационной системы.